

2026-04-26-jackett-autoconfig-clean-rebuild-design

Jackett Auto-Configuration + Clean-Slate Rebuild — Design

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

Date: 2026-04-26 **Author:** brainstorming session (Claude + project owner) **Status:** approved, ready for implementation plan

Implementation skill next: superpowers:writing-plans

1. Problem & Intent

The project owner wants three things in one pass:

1. **Clean-slate rebuild** of the Python-profile container stack so the system boots from a known-zero state.
2. **Full Jackett indexer integration** — every credentialed tracker we have an `.env` triple for should be auto-configured in Jackett at proxy startup, with no manual Jackett UI clicks. New trackers added to `.env` later should pick themselves up automatically.
3. **Comprehensive tests + challenges** at every CONSTITUTION-mandated layer, plus a parity audit so the eventual Go-only flip doesn't silently drop features.

Closing instruction: commit and push to all upstreams as work progresses; hand off when the stack is verified and ready for manual testing.

2. Decisions Locked During Brainstorming

#	Question	Choice	Implication
Q1	What does “all Jackett trackers” mean?	(c) Auto-enable + credential injection from .env	We write code that configures Jackett on the user’s behalf for indexers we have creds for. We do not auto-enable Jackett’s 600+ public indexers blindly.
Q2	Which credentialed indexers?	(b) Generic env discovery + fuzzy match + override	RuTracker, Kinozal, NNMClub, IPTorrents today; any future <NAME>_USERNAME/_PASSWORD/_COOKIES triple gets picked up automatically; JACKETT_INDEXER_MAP resolves ambiguous matches.
Q3	What gets wiped on “clean slate”?	(b) Containers + images + ./config/jackett	Keep ./config/qBittorrent, ./tmp, /mnt/DATA. Forces auto-config to bootstrap from zero, which is the test we want.
Q4/Q5	Python or Go for the rebuild?	(d) Python now, parity audit doc, Go port deferred	Implementation runs against Python proxy. A docs/migration/PARITY_GAPS.md deliverable catalogs what’s still unported in Go so the eventual flip loses nothing.
Q6	Test coverage floor?	(a) Full CONSTITUTION compliance	All 7 layers: unit, integration, E2E, security/penetration, benchmark, automation (mapped

#	Question	Choice	Implication
			to tests/contract/ — see §5), challenge.
Q7	When does push happen?	(c) Per-layer commit + push	After each test layer goes green, commit + push to all 3 remotes.
Approach	Module shape?	(1) Module-in-merge_service, idempotent, startup-time	Lowest topology change, highest testability, easy to lift-and-shift to Go later.

3. Architecture

3.1 Module

New file: download-proxy/src/merge_service/jackett_autoconfig.py

Public API (single function):

```

async def autoconfigure_jackett(
    jackett_url: str,
    api_key: str,
    env: Mapping[str, str],
    indexer_map_override: dict[str, str] | None = None,
    timeout: float = 10.0,
) -> AutoconfigResult: ...

```

AutoconfigResult is a Pydantic model with: - ran_at: datetime - discovered_credentials: list[str] — tracker names found in env (NEVER includes the values) - matched_indexers: dict[str, str] — env name → Jackett indexer ID - configured_now: list[str] — indexer IDs configured this run - already_present: list[str] — indexer IDs Jackett already had configured (idempotency hits) - skipped_no_match: list[str] — env names where fuzzy match failed - skipped_ambiguous: list[dict] — env name + candidate IDs where ≥ 2 indexers matched - errors: list[str] — structured error codes (e.g., jackett_unreachable, jackett_auth_failed, catalog_parse_failed, indexer_config_failed:<id>)

The function never raises out of its boundary; all failures land in errors.

3.2 Call site

download-proxy/src/main.py — after JACKETT_API_KEY is verified non-empty (the existing startup already waits for Jackett’s key extraction), and before the FastAPI app starts serving traffic. Failure is logged WARNING; **boot continues**. Auto-config is best-effort.

3.3 Read endpoint

GET /merge/jackett/autoconfig/last — returns the cached AutoconfigResult from the most recent run. Read-only. Lives in download-proxy/src/api/routes.py. Response shape:

```
{
  "ran_at": "2026-04-26T14:23:11Z",
  "discovered": ["RUTRACKER", "KINOZAL", "NNMCLUB", "IPTORRENTS"],
  "configured_now": ["rutracker", "kinozalbiz"],
  "already_present": ["nnmclub"],
  "skipped_no_match": ["IPTORRENTS"],
  "skipped_ambiguous": [],
  "errors": []
}
```

Credentials NEVER appear in the response, the cached object’s `__repr__`, or any log line at any level.

3.4 Discovery algorithm (3 passes)

1. **Env scan** — regex `^([A-Z][A-Z0-9_]+?)_(USERNAME|PASSWORD|COOKIES)$` over `os.environ`. Group by tracker name. A “credential bundle” is valid when the bundle contains either (USERNAME && PASSWORD) or COOKIES. Names matching the **denylist** (default QBITTORRENT, JACKETT, WEBUI, PROXY, MERGE, BRIDGE, overridable via JACKETT_AUTOCONFIG_EXCLUDE comma-separated env var) are dropped before pass 2 — these are project-internal credentials (qBit auth, Jackett’s own key, etc.) that are guaranteed not to be tracker creds.
2. **Indexer match** — for each surviving bundle:
 - Lookup tracker name in `indexer_map_override` first (highest precedence).
 - If unmatched, fuzzy-match using `Levenshtein.ratio()` (threshold ≥ 0.85 , case-insensitive) against Jackett’s `/api/v2.0/indexers/all/results` catalog.
 - Ambiguous matches (≥ 2 indexers $\geq$ threshold) are skipped with a clear log + entry in `skipped_ambiguous`.

3. Configure — for each matched indexer:

- GET /api/v2.0/indexers (already-configured list). If indexer ID is present, add to already_present, skip.
- GET /api/v2.0/indexers/{id}/config to fetch the field template Jackett expects.
- Map our credential fields onto the template (e.g., username → username, password → password, cookieheader → COOKIES value).
- POST /api/v2.0/indexers/{id}/config with the populated template. On 4xx, log + add to errors. On 5xx, one retry with 2s backoff via existing tenacity.

3.5 No new dependencies

Levenshtein, aiohttp, pydantic, tenacity are already in download-proxy/requirements.txt.

4. Data Flow

```
container boot
└─> start-proxy.sh waits for JACKETT_API_KEY in env
    └─> python -m main
        └─> startup hook: autoconfigure_jackett(...)
            └─ scan os.environ [pass]
1]
    └─ GET /api/v2.0/indexers/all/results
(catalog)
    └─ fuzzy-match bundles → indexer IDs [pass]
2]
    └─ GET /api/v2.0/indexers
(already-configured)
    └─ GET /api/v2.0/indexers/{id}/config
(template, per indexer)
    └─ POST /api/v2.0/indexers/{id}/config (apply,
per indexer)
        └─ build AutoconfigResult, log summary,
            store in module-level cache
        └─> FastAPI app starts, exposes:
            GET /merge/jackett/autoconfig/last
(cached, redacted)
    └─> proxy + merge service serve traffic normally
```

Persistence: Jackett owns the durable state on disk (./config/jackett/Jackett/Indexers/*.json). Auto-config issues API calls; restart-survival is Jackett's job; the idempotency check on next boot prevents double-config.

Result caching: in-process, module-level. Lost on restart; re-run on next boot. No new files, no DB.

Logging: structured (existing project logger). One INFO summary per run (autoconfigured 3/4 discovered indexers). Per-indexer DEBUG. WARNING on errors. **No credential value at any level, ever.**

5. Test Matrix (CONSTITUTION 7-layer floor)

#	Layer	File	Real infra?
1	Unit	tests/unit/merge_service/ test_jackett_autoconfig.py	Mocks via unittest.mock.AsyncMock/ MagicMock (matches existing convention in tests/unit/ test_private_tracker_search allowed in unit only)
2	Integration	tests/integration/ test_jackett_autoconfig_real.py	Real Jackett container via pytest-docker
3	E2E	tests/e2e/ test_jackett_autoconfig_e2e.py	Full stack (podman compose up)
4	Security/ penetration	tests/security/ test_jackett_autoconfig_secrets.py + bandit static scan	Real stack + log files
5	Benchmark	tests/benchmark/ test_jackett_autoconfig_perf.py	Real Jackett or recorded responses (allowed for benchmark stability)
6	Automation	tests/contract/ test_jackett_autoconfig_contract.py (Schemathesis)	Live FastAPI
7	Challenge	challenges/scripts/ jackett_autoconfig_clean_slate.sh	Full live system

5.1 Layer-by-layer assertions

Unit (1): - Env-scan regex correctness across edge cases (lowercase ignored, mixed case, multi-underscore tracker names). - Fuzzy threshold behavior — at, above, and below 0.85. - JACKETT_INDEXER_MAP override precedence over fuzzy match. - Idempotency check skips

already-configured indexers. - `AutoconfigResult.__repr__()` and `.model_dump_json()` redact passwords/cookies. - Errors are caught at function boundary — `autoconfigure_jackett()` never raises.

Integration (2): - Spin up a real Jackett container (pytest-docker fixture). - Inject test env vars (`TESTTRACKER_USERNAME=u`, `_PASSWORD=p`). - Call `autoconfigure_jackett()`, assert it discovered the env, fuzzy-matched a public indexer Jackett ships with, configured it. - Second call: indexer in `already_present`. - `/merge/jackett/autoconfig/last` returns the live result.

E2E (3): - Tear down stack, wipe `./config/jackett`, set fake creds in env, podman compose up, wait for healthchecks. - Poll `/merge/jackett/autoconfig/last` until populated. - Assert expected indexer set configured. - Run a search through the merge service. - Assert at least one Jackett-sourced result appears in the dedup output.

Security (4): - Boot stack with creds in env. Capture stdout/stderr from boot. - Force errors (invalid Jackett key, malformed env) and capture exception tracebacks. - Assert: no credential value appears in boot logs, FastAPI access logs, `/merge/jackett/autoconfig/last` response, exception tracebacks, or `./config/jackett/Jackett/log.txt`. - Run bandit against `merge_service/jackett_autoconfig.py`. Zero high-severity findings.

Benchmark (5): - pytest-benchmark baselines: - Env-scan over 1k env vars. - Fuzzy match against a 50-indexer catalog. - Full happy-path `autoconfigure` round-trip. - Baselines committed to `tests/benchmark/baselines/jackett_autoconfig.json`. - Fail if any benchmark regresses > 2× baseline.

Automation/Contract (6): - Schemathesis loads the OpenAPI schema for `/merge/jackett/autoconfig/last`. - Fuzzes parameters; asserts no 5xx, no schema violations. - Documented mapping: CONSTITUTION says “automation” but `tests/` has no `automation/` directory. We map to `tests/contract/` (the closest existing equivalent — Schemathesis-driven contract testing of the public API surface). This mapping is recorded here so future readers don’t search for a missing dir.

Challenge (7): - `challenges/scripts/jackett_autoconfig_clean_slate.sh`: 1. `podman compose down --remove-orphans` 2. `rm -rf ./config/jackett` 3. `podman compose up -d` 4. Wait for healthchecks (timeout 3 min). 5. Poll `/merge/jackett/autoconfig/last` until `ran_at` populated. 6. Assert configured indexer count ≥ 1 . 7. Run a

search via the merge service API. 8. Assert results contain at least one Jackett-sourced entry. 9. Exit 0 on success, non-zero with diagnostics on failure.

5.2 Resource caps

CLAUDE.md mandatory #9 caps tests at 30-40% host. All test invocations wrapped:

```
nice -n 19 ionice -c 3 -p 1 python3 -m pytest ...
```

Container `mem_limit` already enforces in-container caps. `pytest -p no:randomly --maxfail=1 -x` for deterministic per-layer runs.

6. Error Handling

Failure	Behavior	Visibility
Jackett unreachable at startup	Log WARNING, errors=["jackett_unreachable"], boot continues	/merge/jackett/autoconfig/last shows the error
Jackett 401 (bad/missing key)	Log WARNING jackett_auth_failed, no retries	Same
Catalog response malformed	Log WARNING catalog_parse_failed, boot continues	Same
Credential triple incomplete (USERNAME only)	Skip silently at DEBUG — env may have unrelated vars	DEBUG only
Fuzzy match ambiguous	Skip with WARNING listing candidates, suggest JACKETT_INDEXER_MAP	skipped_ambiguous entry
Fuzzy match below threshold	Skip with INFO	skipped_no_match entry
JACKETT_INDEXER_MAP references unknown indexer ID	Log WARNING, skip mapping	errors entry
Indexer config POST 4xx	Log status + indexer ID (no body)	errors entry
Indexer config POST 5xx	One retry (2s backoff via tenacity), then give up	errors entry

Failure	Behavior	Visibility
Indexer already configured	Skip silently	already_present
Network timeout mid-call	Treated as 5xx for that one indexer; others continue	Per-indexer error

Universal invariants: - `autoconfigure_jackett()` never raises out of its boundary. - Total runtime hard-capped at 60s via outer `asyncio.wait_for`. - Credentials never appear in logs, error messages, the read endpoint, or exception text.

7. Parity Audit Deliverable

File: docs/migration/PARITY_GAPS.md

Method: side-by-side read of `download-proxy/src/{api,merge_service,ui,config}` vs `qBittorrent-go/internal/{api,service,client,models,middleware,config}`. One row per Python public function / endpoint / behavior. No code changes. Estimated 1-2 hours.

Document shape (excerpt):

Python → Go Parity Gaps

Last audited: 2026-04-26 (commit `<sha>`)
 Audit method: side-by-side read of public API surface; no behavior testing.

Summary

- N Python features audited
- X fully ported
- Y partial
- Z missing

Matrix

Python module	Feature	Go location	Status
		Risk if Go-only today	
----- ----- ----- -----			
<code>`merge_service/search.py`</code>	<code>`start_search()`</code>	orchestrator	
	<code>`internal/service/merge_search.go`</code>	Ported	low
<code>`merge_service/enricher.py`</code>	TMDB title resolution	(none)	
	Missing results lack posters/years		
<code>`merge_service/validator.py`</code>	BEP 48 HTTP scrape	(none)	
	Missing no peer-count validation		
...	...	...	...

Per-gap follow-up specs (proposed, in priority order)

1. Port ``enricher.py`` to ``internal/service/enricher.go``
2. Port ``validator.py`` (BEP 48 + BEP 15)
3. ...

Status definitions (locked): - **Ported** — feature exists in Go with equivalent behavior. - **Partial** — feature exists in Go but lacks a sub-behavior. - **Missing** — no Go counterpart.

8. Verification & Push Protocol

Order of operations (each step is a gate; failure halts):

#	Step	Commit + push?
1	Pre-flight: clean tree, env vars present	—
2	Tear down: containers + images + ./ config/ jackett. Keep ./config/ qBittorrent, ./ tmp, /mnt/DATA	—
3	Implement auto-config module + wire into main.py + add endpoint	—
4	Layer 1 (Unit) green	yes
5	Boot real stack (podman compose up -d), wait for healthchecks	—
6	Layer 2 (Integration) green	yes
7	Layer 3 (E2E) green	yes

#	Step	Commit + push?
8	Layer 4 (Security) green	yes
9	Layer 5 (Benchmark) green, baselines committed	yes
10	Layer 6 (Automation/ Contract) green	yes
11	Layer 7 (Challenge) script exits 0	yes
12	Parity audit: read both codebases, write PARITY_GAPS.md	yes
13	Final clean- slate verification: full tear-down + boot from zero, run challenge again, capture output into commit message	yes
14	Handoff message to project owner	—

Push target: `git push origin <branch> && git push github <branch> && git push upstream <branch>` after each gate. (All three URLs identical — push is idempotent on the remote, but we do all three so refs match locally.)

Hard stops (require owner input): - Any layer fails after one fix attempt. - Any push rejected. - Healthcheck unhealthy after 3 minutes. - Bench regression > 2× baseline.

9. File Inventory

9.1 New files

- `download-proxy/src/merge_service/jackett_autoconfig.py` — the module
- `tests/unit/merge_service/test_jackett_autoconfig.py`
- `tests/integration/test_jackett_autoconfig_real.py`
- `tests/e2e/test_jackett_autoconfig_e2e.py`
- `tests/security/test_jackett_autoconfig_secrets.py`
- `tests/benchmark/test_jackett_autoconfig_perf.py`
- `tests/benchmark/baselines/jackett_autoconfig.json`
- `tests/contract/test_jackett_autoconfig_contract.py`
- `challenges/scripts/jackett_autoconfig_clean_slate.sh` (executable)
- `challenges/scripts/run_all_challenges.sh` (currently no challenges/ dir)
- `docs/migration/PARITY_GAPS.md`
- `docs/superpowers/specs/2026-04-26-jackett-autoconfig-clean-rebuild-design.md` (this file)

9.2 Modified files

- `download-proxy/src/main.py` — call `autoconfigure_jackett()` after Jackett key extraction, before uvicorn
- `download-proxy/src/api/routes.py` — add `GET /merge/jackett/autoconfig/last`
- `download-proxy/src/api/__init__.py` — register the new route if needed
- `docs/JACKETT_INTEGRATION.md` — append “Auto-Configuration” section
- `CLAUDE.md` — add `JACKETT_INDEXER_MAP` and `JACKETT_AUTOCONFIG_EXCLUDE` to the env vars line; note the auto-config endpoint
- `AGENTS.md` — same env additions
- `.env.example` (if exists) — add `JACKETT_INDEXER_MAP=` and `JACKETT_AUTOCONFIG_EXCLUDE=` placeholders

9.3 Untouched

- `qBitTorrent-go/**` (parity audit reads but does not modify)
- `frontend/**`
- `plugins/**`
- `webui-bridge.py`
- `start-proxy.sh` (unless implementation discovers a timing tweak is needed)

9.4 Bugfix log

Per CLAUDE.md mandatory #10 + CONST-032: any bug discovered during this work goes into `docs/issues/fixed/BUGFIXES.md` with a corresponding `challenges/scripts/jackett_autoconfig_<bug-slug>.sh` regression guard.

10. Out of Scope (explicit)

- Porting any Python feature to Go. The parity doc *catalogs* the gap; closing it is future work in dedicated specs.
- Go-profile rebuild or Go-side Jackett auto-config.
- Wiping `./config/qBittorrent/` or `/mnt/DATA`.
- Manual testing scripts — handoff is verbal.
- Auto-enabling Jackett's public indexer set without credentials. The Q1 decision was credentialed-only.

11. Open Questions for Implementation Plan

(These do not block design approval; they get resolved when writing-plans produces the step-by-step plan.)

- Exact branch name for this work (suggest `jackett-autoconfig-clean-rebuild`).
- Whether the existing test stack already auto-starts Jackett via `pytest-docker`, or whether the integration layer needs a new fixture.
- Whether `docs/issues/fixed/BUGFIXES.md` exists — if not, it gets created on first bugfix.
- Whether `start-proxy.sh` injects `JACKETT_API_KEY` to the proxy container before `python -m main` runs (it should, but verify at implementation time).